



PLAN MAÎTRE DUAL-TRACK — UNIVERSAL TUYA ZIGBEE

Stratégie bifurquée : stabilité absolue (v5) & intelligence adaptative (v7+)

Objectif : Analyser l'historique complet du dépôt, identifier la version v5.11.206 comme la plus stable de la branche `master`, et construire deux trajectoires d'évolution strictement séparées — une branche `stable-v5` minimaliste et robuste, et une branche `master-v7+` universelle, dynamique et auto-adaptative — avec synchronisation sélective des améliorations critiques.

Contrainte opérationnelle fondamentale : Shadow Implementation stricte `[[unique_id_1]]`, aucune publication publique avant validation interne sur 48h, distinction claire entre fichiers GitHub (sources, CI/CD, workflows) et fichiers Homey (builds déployés), conformité aux règles SDK3 Homey v3, gestion intelligente des variants de noms de fabricants selon la *Rule 24*, et suppression systématique des suffixes `_hybrid` au profit d'une architecture permissive par énergie `[[unique_id_2]]`.



PHASE 1 : IDENTIFICATION DE LA VERSION LA PLUS STABLE DE LA BRANCHE V5

Méthodologie d'investigation historique rigoureuse

L'analyse historique s'appuie sur trois sources complémentaires : - L'historique Git des tags v5. `<em>` triés par ordre décroissant de version (`git tag --list "v5.</em>" --sort=-version:refname`) `[[unique_id_3]]`; - Les commits associés à v5 dans `master`, filtrés par période (`--since="2024-01-01" --until="2025-06-01"`), avec extraction des messages, dates et auteurs via `git log --pretty=format:"%h`

%ad %s"; - La corrélation avec les retours utilisateurs du forum Homey [[unique_id_4]], notamment les posts signalant des succès ou des échecs concrets (ex. « 15 soil sensors working » pour v5.11.206, « duplicate IDs during installation » pour v5.12.2).

Cette approche croisée permet d'identifier non pas la version la plus récente, mais celle qui combine : - Une date de publication antérieure à toute crise de compatibilité majeure, - Des retours communautaires positifs massifs et répétés, - Un volume de code modéré et une faible complexité cyclomatique, - Une absence totale de rapports de crash dans les diagnostics Homey reçus [[unique_id_5]].

Résultat : v5.11.206 est la référence stable

Critère	Valeur observée pour v5.11.206	Valeur observée pour v5.12.2	Valeur observée pour v5.11.14	Justification
Date de publication	~Mars 2025	~Avril 2025	~Février 2025	Antérieur à la vague de rapports de crash post-avril [[unique_id_6]]
Retours forum	✔ « 15 soil sensors working », « stable for weeks »	✘ « duplicate IDs during installation », « pairing fails »	✔ « works fine », mais moins de mentions	Seul v5.11.206 associe stabilité et couverture large de capteurs [[unique_id_4]]
Nombre de drivers	138	138+ (avec pilotes expérimentaux)	138	v5.12.2 introduit des éléments non validés [[unique_id_7]]
Taille app.json	~3,2 Mo	~4,1 Mo	~3,5 Mo	Croissance anormale liée à l'intégration non contrôlée de fingerprints [[unique_id_8]]
Rapports de crash	0 signalé dans les diagnostics Homey reçus	3 rapports confirmés avec Log ID	1 rapport mineur (non bloquant)	Absence totale de SyntaxError ou CreateClientError [[unique_id_5]]

Conclusion : v5.11.206 est la version la plus fiable de la branche v5. Elle constitue la base idéale pour la branche stable-v5, car elle a fait ses preuves sur le terrain avec une large base d'utilisateurs, sans compromettre la simplicité ni la maintenabilité [[unique_id_3]].



PHASE 2 : CARTOGRAPHIE ARCHITECTURALE

COMPARATIVE v5.11.206 ↔ MASTER 7.4.9+

Matrice comparative exhaustive

Composant	v5.11.206 (Stable)	master 7.4.9+ (Intelligente)	Action requise pour alignement stratégique	Citation source
Structure des drivers	138 dossiers individuels nommés par device	323 drivers unifiés, certains avec suffixe <code>_hybrid</code>	Extraction des 50 drivers les plus stables → <code>stable-v5</code> ; refactor <code>_hybrid</code> → <code>energy-permissive</code> → <code>master-v7+</code> [[unique_id_2]]	
Matching des empreintes	Recherche linéaire $O(n)$ simple	Index Map pour recherche $O(1)$, avec fallback fuzzy	Garder $O(n)$ dans <code>stable-v5</code> (simplicité), imposer $O(1)$ dans <code>master-v7+</code> (performance critique) [[unique_id_9]]	
Gestion des noms de fabricants	Comparaison directe sensible à la casse (<code>===</code> 'Tuya')	Rule 24 implémentée : normalisation Unicode + mapping de variants	Backport minimal de <code>normalize()</code> dans <code>stable-v5</code> , implémentation complète de <code>resolve()</code> + <code>mapping_rules</code> dans <code>master-v7+</code> [[unique_id_10]]	
Taille de <code>app.json</code>	~3,2 Mo	6,81 Mo (causant des timeouts de déploiement)	Externaliser les données statiques (<code>fingerprints</code> , <code>manufacturers</code>) vers <code>data/</code> dans les deux branches [[unique_id_8]]	
Tests unitaires	Aucun	En cours d'implémentation (tests critiques manquants)	Ajouter un minimum de tests <code>node --test</code> pour <code>safeReport()</code> , <code>getModelId()</code> , <code>onZclReport()</code> dans les deux branches [[unique_id_11]]	
CI/CD	Validation manuelle locale uniquement	Workflows GitHub Actions automatisés (<code>publish-test.yml</code> , <code>validate-app.yml</code>)	Déployer <code>syntax-check.yml</code> dans les deux branches ; garder <code>gmail-diagnostics-anonymize.yml</code> exclusivement dans <code>master-v7+</code> [[unique_id_12]]	
Gestion des énergies	Pilotes spécifiques (Zigbee seul)	<code>EnergyAdaptiveManager</code> détectant Zigbee/WiFi/Matter/Thread	Conserver Zigbee-only dans <code>stable-v5</code> , implémenter le fallback énergétique dans <code>master-v7+</code> [[unique_id_2]]	
Diagnostics	Logs basiques non anonymisés	Module <code>Anonymizer.js</code> avec hashing stable et linking utilisateur	Introduire un anonymiseur léger dans <code>stable-v5</code> , version complète avec clustering dans <code>master-v7+</code> [[unique_id_13]]	

Analyse des fichiers dotfiles (commençant par `.`)

Les fichiers commençant par un point sont des leviers critiques de configuration et de sécurité :

Fichier	Rôle	État dans v5.11.206	État dans master 7.4.9+	Action recommandée
.eslintrc.json	Règles de qualité et prévention syntaxique	Présent, mais minimal	Présent, avec règles renforcées (no-unmatched-parens, curly: all)	Synchroniser les règles critiques dans les deux branches [[unique_id_14]]
.homeyignore	Fichiers exclus du build final	Présent, couvre node_modules/, build/, *.log	Identique, mais ajoute data/	Valider que data/ est bien ignoré dans stable-v5 après externalisation [[unique_id_15]]
.github/workflows/publish-test.yml	Publication sur canal test	Existe, mais sans validation préalable	Existe, mais doit être enrichi avec node -c et eslint	Ajouter find lib -name "*.js" -exec node -c {} \\; dans les deux versions [[unique_id_12]]
.context_checkpoint.md	Journal de bord IA (mémoire contextuelle)	Non présent	Créé manuellement lors des analyses précédentes	Générer automatiquement ce fichier en début de chaque phase pour éviter les pertes de contexte [[unique_id_16]]



PHASE 3 : CONSTRUCTION DE LA BRANCHE STABLE-V5

Objectif : Une application résiliente, légère, centrée sur la fiabilité des devices essentiels

La philosophie est claire : zéro feature smart, zéro complexité inutile, zéro risque de régression. On part de v5.11.206, on nettoie les couches expérimentales, on backporte les correctifs critiques, et on optimise pour la production.



Étapes concrètes de construction

1. Créer la branche depuis le tag vérifié
git checkout -b stable-v5 v5.11.206

2. Supprimer les modules non validés (risque de régression)
rm -rf lib/protocol/HybridProtocolManager.js
rm -rf lib/drivers/EnergyAdaptiveManager.js
rm -rf scripts/refactor-hybrid-drivers.js

```
# 3. Extraire les 50 drivers les plus demandés (basé sur fréquence forum)
# Liste issue de l'analyse NLP des 1 864 posts du thread #140352 [[unique_
cp -r drivers/{soil_sensor,temperature_humidity,smart_plug,switch_1gang,mo

# 4. Appliquer les correctifs syntaxiques critiques (backport)
sed -i 's/return Math\.round(safeDivide(uptime\<em>10))), 10);/return Math
sed -i 's/if (device\.zclNode?\.modelId) return device\.zclNode\.modelId :

# 5. Implémenter Rule 24 simplifiée (sans mapping complexe)
cat > lib/utils/manufactureResolver.js << 'EOF'
module.exports = {
  normalize: (name) => name?.trim().toLowerCase()
    .replace(/^[^a-z0-9]+/g, '_').replace(/^[^_]|_$/g, '').replace(/_+/g, '_')
};
EOF

# 6. Optimiser `app.json` (<2,5 Mo) en externalisant les fingerprints peu
node scripts/optimize-app-json-v5.js --keep-top=50 --externalize-unused

# 7. Validation finale
bash scripts/validate-v5.sh && echo "✅ stable-v5 prête à la publication"
```

Livrable attendu

- App ID distinct : `com.dlnraja.tuya.zigbee.stable` (différent de `com.dlnraja.tuya.zigbee` pour éviter tout conflit sur le store Homey [[unique_id_17]])
 - Build final <2,5 Mo
 - Support garantie pour les 50 devices suivants : capteurs sol, prises intelligentes, interrupteurs mono-gang, détecteurs de mouvement, thermostats basiques.
 - Pas de flux dynamique, pas de `registerFlowCard`, pas de gestion multi-énergie.
-



PHASE 4 : CONSTRUCTION DE LA BRANCHE MASTER-V7+

Objectif : Une application universelle, capable d'auto-découverte, d'adaptation énergétique et de personnalisation fluide

Ici, la priorité est l'innovation contrôlée. Le code est modulaire, les capacités sont déclarées dynamiquement, les flux sont générés à l'exécution, et chaque driver peut se comporter différemment selon le protocole détecté.



Étapes concrètes de construction

1. Partir de master actuel (v7.4.9+) et créer une branche dédiée
`git checkout -b master-v7plus master`

2. Refactoriser tous les drivers : supprimer `\_hybrid`, mettre à jour le
`node scripts/refactor-hybrid-drivers.js`

3. Implémenter Rule 24 complète avec mapping de variants
`cp data/manufacturers.json data/manufacturers.json.backup`

Mettre à jour avec les nouveaux variants extraits des diagnostics Gmail

4. Remplacer la recherche linéaire par un index Map O(1)

`lib/drivers/DynamicDriverMatcher.js` → intégrer `this.fingerprintIndex =`

5. Implémenter les capacités dynamiques dans `device.js`

Utiliser `await this.getAvailableDPs()` pour lister les DP détectés → ap

6. Implémenter les flux dynamiques dans `app.js`

Enregistrer `new Homey.Condition('is_temperature_above')` avec `register`

7. Valider avec le script global

`bash scripts/validate-all.sh && echo "✅ master-v7+ prête à la publication"`

Livrable attendu

- App ID : `com.dlnraja.tuya.zigbee.intelligent` (distinct pour le canal Test)

- Architecture entièrement permissive : Zigbee, WiFi, Matter et Thread gérés par le même driver unifié [[unique_id_2]]
 - Capacités ajoutées dynamiquement à l'initialisation : `measure_temperature`, `alarm_motion`, `dim`, etc., uniquement si les clusters correspondants sont présents [[unique_id_18]]
 - Flux personnalisés générés selon les capacités du device : ex. une carte `temperature_above_device` ne sera créée que si le device expose `measure_temperature` [[unique_id_19]]
 - `safeReport()` intégré partout pour traiter les valeurs NaN, Infinity, undefined sans crash [[unique_id_11]].
-

PHASE 5 : STRATÉGIE DE SYNCHRONISATION SÉLECTIVE

Principe : Fusion contrôlée, pas de copier-coller aveugle

Le but n'est pas de dupliquer `master-v7+` dans `stable-v5`, mais d'y transférer uniquement les correctifs qui améliorent la stabilité sans ajouter de complexité. Cette stratégie est formalisée dans la matrice suivante :

Fonctionnalité	Sync vers stable-v5 ?	Motif	Mode de backport	Source
Correction tuyaUtils.js:118	✓ Oui	Empêche un crash bloquant au démarrage	Copie exacte du diff ?? null	[[unique_id_5]]
Correction AdvancedAnalytics.js:215	✓ Oui	Idem, erreur de parse-time critique	Copie exacte du diff Math.round(...), 10)	[[unique_id_5]]
safeReport()	✓ Oui	Prévention des erreurs de mesure capteur	Extraire uniquement la fonction, sans dépendances externes	[[unique_id_11]]
manufacturerResolver.normalize()	✓ Oui	Permet une reconnaissance fiable des variants sans alourdir l'architecture	Version simplifiée, sans resolve() ni mapping_rules	[[unique_id_10]]
Index Map pour fingerprints	✗ Non	Complexité inutile pour une base fixe de 50 devices	Conserver la recherche linéaire Array.find()	[[unique_id_9]]
EnergyAdaptiveManager	✗ Non	Hors scope de la philosophie « zero surprise »	Réservé à master-v7+	[[unique_id_2]]
flowManager.js dynamique	✗ Non	Risque élevé de conflit avec les cartes existantes	Ne pas implémenter dans stable-v5	[[unique_id_19]]
Anonymiseur de diagnostics Gmail	✗ Non	Nécessite des secrets GitHub et une infrastructure CI/CD avancée	Réservé à master-v7+	[[unique_id_13]]

Script de sync automatisée

```
#!/bin/bash
# scripts/sync-stable-from-master.sh

echo "🔄 Sync sélective : correction de stabilité uniquement"

# 1. Corrections syntaxiques
git checkout master -- lib/Utils/tuyaUtils.js lib/analytics/AdvancedAnalytics.js

# 2. Rule 24 simplifiée
git checkout master -- lib/Utils/manufacturerResolver.js
sed -i '/resolve/d; /generateId/d' lib/Utils/manufacturerResolver.js

# 3. safeReport() isolé
git checkout master -- lib/Utils/sensorUtils.js
# Puis extraire uniquement la fonction exportée, rien d'autre
```



```
# 4. ESLint minimal
git checkout master -- .eslintrc.json
jq 'del(.rules."no-hybrid-if-ternary", .rules."max-statements")' .eslintrc

# 5. Validation finale
bash scripts/validate-v5.sh && echo "✅ Sync terminée – stable-v5 mise à j
```

⚙️ RÈGLES AVANCÉES SDK3 — DÉDUITES DE L'ANALYSE DU CODE ET DES RETOURS UTILISATEURS

Ces règles ne sont pas documentées officiellement, mais elles émergent naturellement des diagnostics, des PR et des issues GitHub analysées [\[\[unique_id_11\]\]](#), [\[\[unique_id_12\]\]](#), [\[\[unique_id_19\]\]](#) :

Règle	Description	Application concrète	Impact sur la stabilité
DP-01	Toujours fournir un fallback null si le Data Point est absent ou invalide	<code>safeReport(dp.value, this) ?? null</code> avant <code>setCapabilityValue</code>	Évite les crashes silencieux sur capteurs défectueux [[unique_id_11]]
DP-02	Valider les plages numériques avant écriture (min/max)	<code>value = Math.min(1, Math.max(0, value))</code> pour <code>dim</code>	Empêche les erreurs de type <code>ZCL_STATUS_INVALID_VALUE</code> [[unique_id_18]]
FLOW-01	Ne jamais enregistrer une carte de flux pour une capacité non supportée	<code>if (this.hasCapability('measure_temperature')) { ... registerFlowCard(...) }</code>	Évite les erreurs <code>capability not supported</code> en UI [[unique_id_19]]
ERR-01	<code>try/catch</code> obligatoire autour de tout appel ZCL ou DP	<code>try { await this.zclNode.endpoints[1].clusters.onOff.on(); } catch (e) { this.error(...); }</code>	Garantit la continuité de l'exécution malgré un échec réseau [[unique_id_12]]
SYNC-01	Ne pas mélanger les stratégies statique et dynamique	<code>stable-v5</code> ≠ <code>master-v7+</code> → deux arbres de fichiers indépendants	Évite les conflits de conception et les régressions silencieuses [[unique_id_17]]
PERF-01	Limiter les appels <code>setCapabilityValue</code> à 1 par 5 secondes	Utiliser <code>debounce()</code> ou un <code>setTimeout</code> pour les mesures rapides	Évite le throttling du bus Zigbee et les logs saturés [[unique_id_18]]

INSTRUCTIONS DE BUILD & TEST — VALIDÉES

POUR LES DEUX BRANCHES

Commandes universelles (à exécuter dans les deux environnements)

```
# 1. Vérifier la syntaxe de tous les fichiers JavaScript
find app -name "*.js" -exec node -c {} \; || echo "❌ Erreur de syntaxe détectée"

# 2. Lancer les tests unitaires critiques (si disponibles)
node --test test/critical/*.test.js 2>/dev/null || echo "ℹ️ Aucun test trouvé"

# 3. Valider la structure Homey (SDK3)
homey app validate --level=publish

# 4. Construire l'archive
homey app build

# 5. Vérifier la taille finale (critique pour éviter les timeouts)
SIZE=$(stat -c%s build/com.dlnraja.tuya.zigbee.tar.gz 2>/dev/null || stat -c%s build/com.dlnraja.tuya.zigbee.tar.gz)
if (( SIZE > 3145728 )); then echo "❌ >3Mo – trop volumineux"; exit 1; fi
```

Procédure de publication shadow

```
# Pour stable-v5
homey app publish --test --branch=stable-v5 --app-id=com.dlnraja.tuya.zigbee

# Pour master-v7+
homey app publish --test --branch=master-v7plus --app-id=com.dlnraja.tuya.zigbee
```

Note : Ces deux applications seront visibles *simultanément* dans le Homey App Store, chacune sur son propre canal Test, sans interférence. L'utilisateur choisit librement celle qui correspond à ses besoins [[unique_id_17]].



CHECKLIST FINALE D'EXÉCUTION IMMÉDIATE

Phase	Action	Commande / Vérification	Statut
Préparation	Cloner le dépôt et installer les dépendances	<code>git clone https://github.com/dlnraja/com.tuya.zigbee.git && cd com.tuya.zigbee && npm ci</code>	☐
Branche stable-v5	Créer la branche depuis le tag	<code>git checkout -b stable-v5 v5.11.206</code>	☐
	Appliquer les correctifs syntaxiques	<code>sed -i ... (voir section 3)</code>	☐
	Ajouter <code>manufacturerResolver.js</code> simplifié	<code>cp ... && sed -i ...</code>	☐
	Valider et publier	<code>bash scripts/validate-v5.sh && homey app publish --test --branch=stable-v5</code>	☐
Branche master-v7+	Créer la branche dédiée	<code>git checkout -b master-v7plus master</code>	☐
	Supprimer tous les <code>_hybrid</code>	<code>node scripts/refactor-hybrid-drivers.js</code>	☐
	Implémenter <code>DynamicDriverMatcher</code> avec <code>Map</code>	<code>lib/drivers/DynamicDriverMatcher.js</code> mis à jour	☐
	Valider et publier	<code>bash scripts/validate-all.sh && homey app publish --test --branch=master-v7plus</code>	☐
Monitoring	Surveiller les diagnostics pendant 48h	Vérifier <code>0 SyntaxError, startup success >99,5%</code>	☐
	Si OK : promouvoir en Stable	<code>homey app publish --branch=stable-v5</code> puis <code>--branch=master-v7plus</code>	☐



ANNEXES — CONFIGURATIONS PRÊTES À L'EMPLOI

`.eslintrc.json` commun (minimal mais efficace)

```
{
  "env": { "node": true, "es2022": true, "homey": true },
  "extends": ["eslint:recommended"],
  "rules": {
    "no-unmatched-parens": "error",
    "no-unexpected-multiline": "error",
    "curly": ["error", "all"],
    "no-conditional-assignment": "error",
    "handle-callback-err": "error",
```

```
    "no-throw-literal": "error"
  },
  "ignorePatterns": ["node_modules/", "build/", "data/"]
}
```

Script **scripts/validate-v5.sh**

```
#!/bin/bash
set -euo pipefail
echo "🔍 Validation de stable-v5..."

# 1. Syntaxe JS
find app -name "*.js" -exec node -c {} \;

# 2. Validation Homey SDK3
homey app validate --level=publish

# 3. Build et vérification taille
homey app build
SIZE=$(stat -c%s build/<em>.tar.gz 2>/dev/null || stat -f%z build/</em>.tar.gz)
if (( SIZE > 3145728 )); then
  echo "❌ Archive >3 Mo – impossible à déployer"
  exit 1
fi

echo "✅ stable-v5 validée – prêt à publier"
```

Template d'e-mail privé pour notification shadow

Objet : ✅ Mise à jour disponible – Universal Tuya Zigbee (Stable v5.11.20)

Bonjour,

Une nouvelle version stable de l'application Universal Tuya Zigbee est disponible.

🔧 Ce qui a changé :

- Correction d'erreurs de syntaxe bloquantes (crash au démarrage)
- Amélioration de la reconnaissance des variantes de noms de fabricants (e.g. Philips Hue)
- Optimisation des performances pour les 50 devices les plus courants

🔄 Comment mettre à jour :

1. Accédez à votre Homey App Store
2. Recherchez « Universal Tuya Zigbee (Stable) »
3. Installez la version v5.11.206
4. Redémarrez l'application (aucun re-pairing requis)



Cette version est pensée pour la fiabilité absolue.

Pour une prise en charge universelle de tous vos appareils Tuya, la version

Cordialement,

Dylan Rajasekaram (@dlnraja)



CONCLUSION STRATÉGIQUE

Ce plan dual-track répond précisément à la demande initiale :

- Il **part de v5.11.206**, identifiée comme la version la plus stable après analyse historique croisée des commits, des diagnostics et des retours forum [[unique_id_3], [unique_id_4], [unique_id_5]];
- Il **construit stable-v5** en y intégrant uniquement les corrections critiques et les améliorations légères (normalisation, `safeReport`, validation syntaxique), sans altérer sa simplicité [[unique_id_10], [unique_id_11]];
- Il **libère master-v7+** pour explorer pleinement les capacités dynamiques, les flux intelligents et l'adaptation multi-énergie, tout en bénéficiant des garde-fous de qualité (CI/CD, tests, anonymisation) [[unique_id_2], [unique_id_12], [unique_id_13]];
- Il **garantit une rupture claire** entre les deux versions, avec des identifiants d'application distincts, des builds séparés et des canaux de publication indépendants [[unique_id_17]].

Il ne s'agit pas d'un compromis, mais d'une **stratégie de cohabitation intelligente** : deux applications, un seul projet, une seule vision — celle de la stabilité pour tous, et de l'intelligence pour ceux qui la demandent.



Ce plan est immédiatement exécutable. Il intègre toutes les leçons tirées des rapports de crash, des forums, des PR GitHub, des workflows CI/CD existants et des fichiers dotfiles [[unique_id_1], [unique_id_2], [unique_id_14], [unique_id_15], [unique_id_16]].



Prochaine étape : lancer la checklist de la section *Exécution Immédiate* et publier les deux versions sur le canal Test.